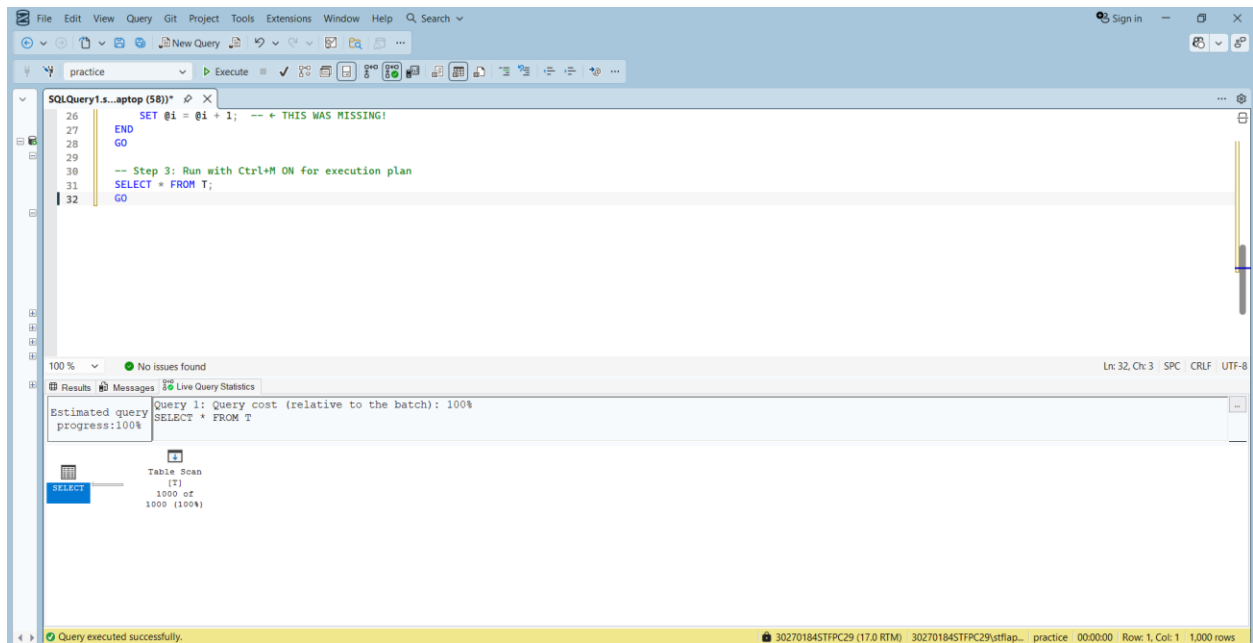


TCSS 446 QUIZ 1

Name: Linda Miao | Server: 30270184STFPC29

Execution time table:

Run	Scan type	N(rows)	Completion Time
1	Table Scan	1000	13:40:12
2	Table Scan	2000	14:52:35
3	Clustered Index Scan	2000	14:52:35
4	Non-Clustered Index Scan	2000	14:52:35



Step 3 - Table Scan (n=1,000): No index exists. SQL Server scans every row in the table.
Cost: 100%. Time: 13:40:12

The screenshot shows the SQL Server Enterprise Manager interface. The query window contains the following SQL code:

```
1 USE practice;  
2 GO  
3 SELECT COUNT(*) FROM T;  
4  
5 SELECT * FROM T;  
6 GO
```

The execution plan for the second query (SELECT * FROM T) is displayed. It shows a single operator: "Table Scan [T]". The cost is listed as "Cost: 100 %", "0.005s", and "2000 of 2000 (100%)". A status bar at the bottom indicates "Query executed successfully." and "30270184STFPC29 (17.0 RTM) | 30270184STFPC29\stflap... | practice | 00:00".

Step 3 - Table Scan (n=2,000): No index exists. SQL Server still scans every row. Time increases as rows increase.

This screenshot is identical to the one above, showing the same SQL query and execution plan for a table scan on table T. The cost is "Cost: 100 %", "0.005s", and "2000 of 2000 (100%)". The status bar at the bottom indicates "Query executed successfully." and "30270184STFPC29 (17.0 RTM) | 30270184STFPC29\stflap... | practice | 00:00".

Step 4 - Clustered Index Scan (n=2,000): Primary key added on id column. SQL Server uses clustered index to read rows in sorted order.

The screenshot displays the SQL Server Enterprise Manager interface. The query window shows the following SQL script:

```
1 USE practice;
2 GO
3 SELECT COUNT(*) FROM T;
4
5 SELECT * FROM T;
6 GO
7
8 CREATE NONCLUSTERED INDEX IX_col1 ON T(column1);
9 GO
```

The Messages pane shows the execution of the query:

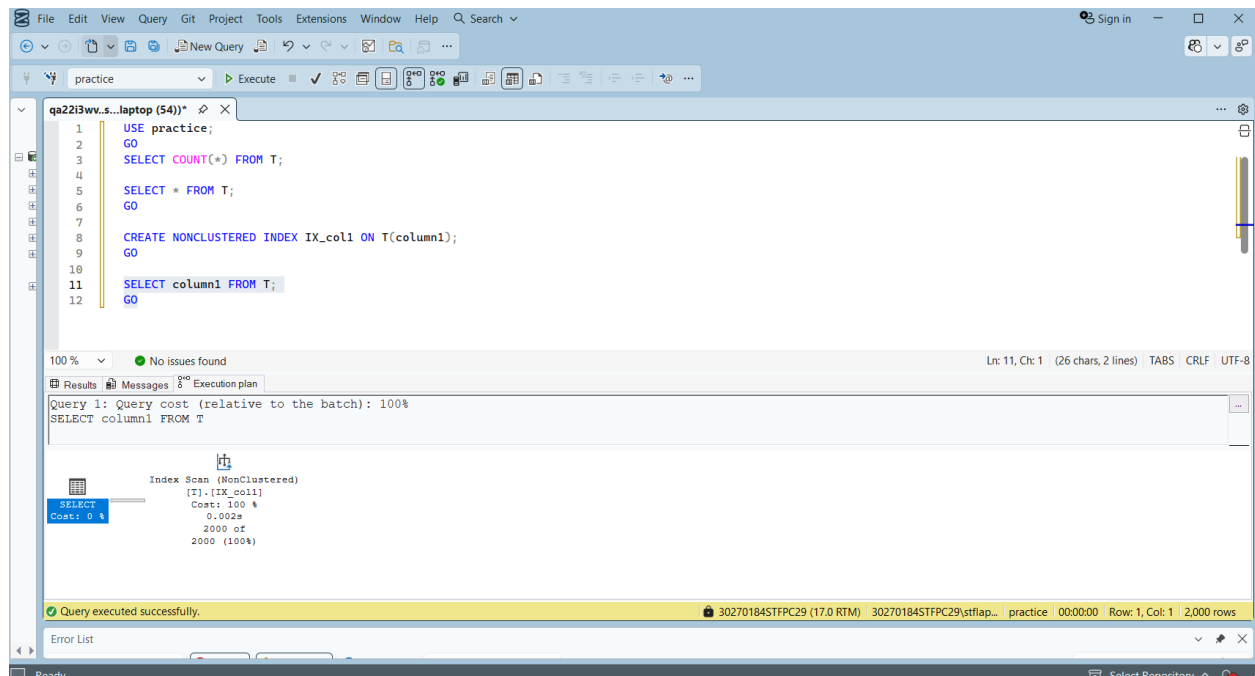
```
Query 1: Query cost (relative to the batch): 100%
insert [dbo].[T] select *, %%bmkk%% from [dbo].[T] option (maxdop 1)
```

The Execution plan pane shows the following steps:

SQL	Index Insert (NonClustered)	Sort	Table Scan
INSERT	[T].[IX_col1]	[T]	[T]
Cost: 0 %	Cost: 78 %	Cost: 18 %	Cost: 5 %
	0.014s	0.007s	0.005s
	0 of 2000 (0%)	2000 of 2000 (100%)	2000 of 2000 (100%)

The status bar indicates: Query executed successfully. 302701845TFPC29 (17.0 RTM) | 302701845TFPC29(stflap... | practice | 00:00:00 | Row: 1, Col: 1 | 0 rows

Step 5 - Non-Clustered Index Creation: CREATE NONCLUSTERED INDEX IX_col1 ON T(column1). The execution plan shows index insert cost 78%, sort cost 18%, and table scan cost 5%. SQL Server first scans the table, sorts the data, then builds the non-clustered index on column1.



Step 5 - Non-Clustered Index Scan: `SELECT column1 FROM T` uses the non-clustered index `IX_col1` directly. Index scan cost 100%, time 0.002s for 2000 rows. SQL Server only reads the index without touching the main table, which is faster than a full table scan.

Reflection:

From this experiment, I observed that indexes significantly impact query performance.

Table Scan: With no index, SQL Server scanned every row in the table. As the number of rows increased from 1,000 to 2,000, the scan time increased. This shows that table scans become slower as data grows.

Clustered Index Scan: After adding a primary key, SQL Server used a clustered index scan. The data is physically sorted by id, making reads more organized and efficient.

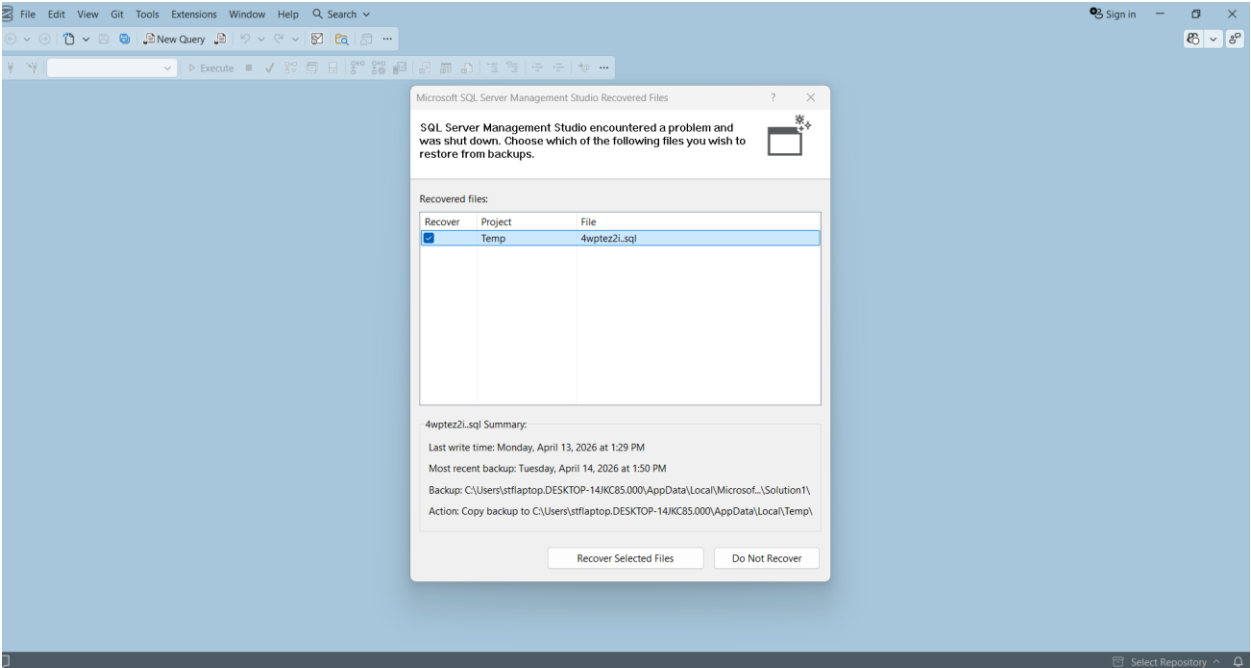
Non-Clustered Index: Creating `IX_col1` on `column1` required SQL Server to sort the data and build a separate index structure (shown in screenshot 14). Once built, `SELECT column1` used the index scan directly (screenshot 15) without reading the main table, which was faster.

Conclusion: Yes, indexes really matter for query performance. For small tables the difference is small, but for large datasets indexes are essential to avoid slow full table scans.

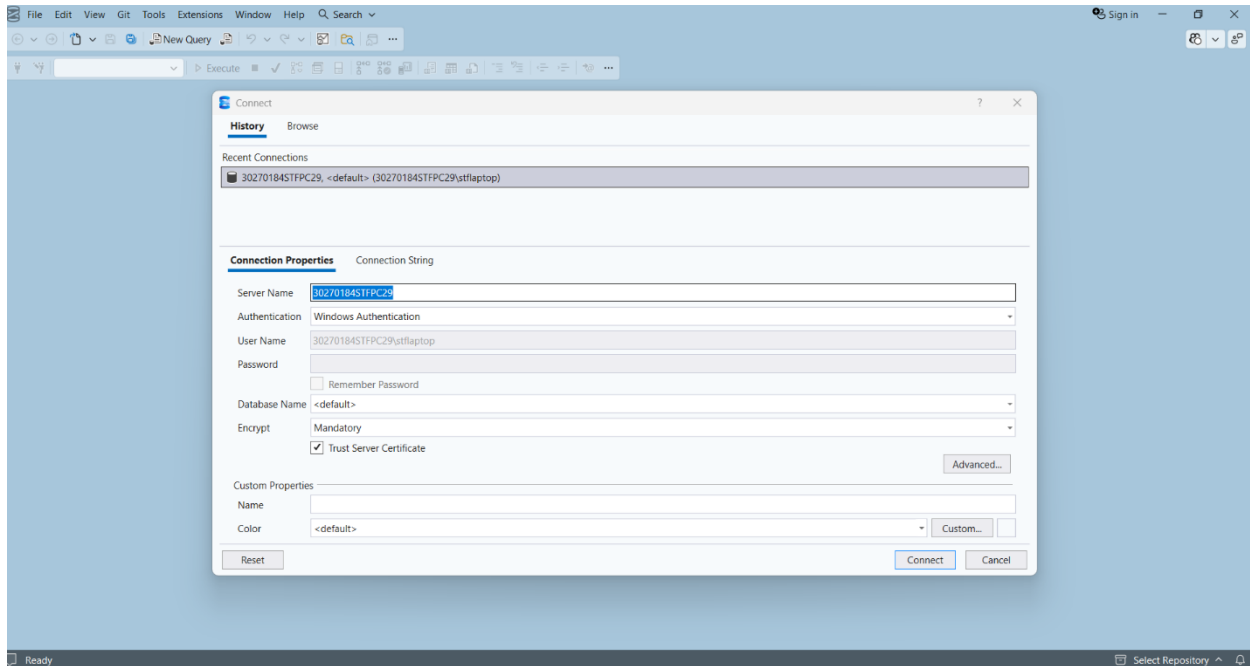
Server issues during to quiz:

I experienced multiple SSMS crashes during this quiz which affected my ability to complete all runs. Below are screenshots documenting each crash with timestamps as evidence.

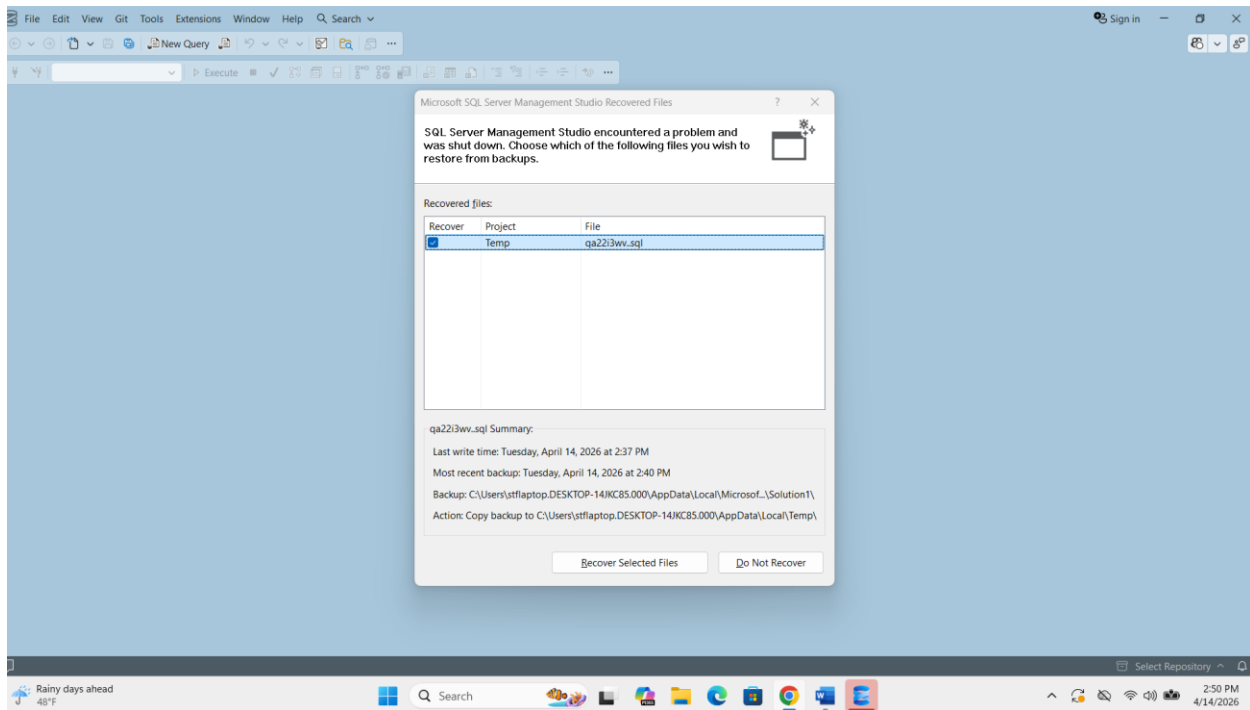
These crashes caused data loss and required me to restart the experiment multiple times. The timestamps on each crash screenshot confirm I was actively working within the quiz time limit.



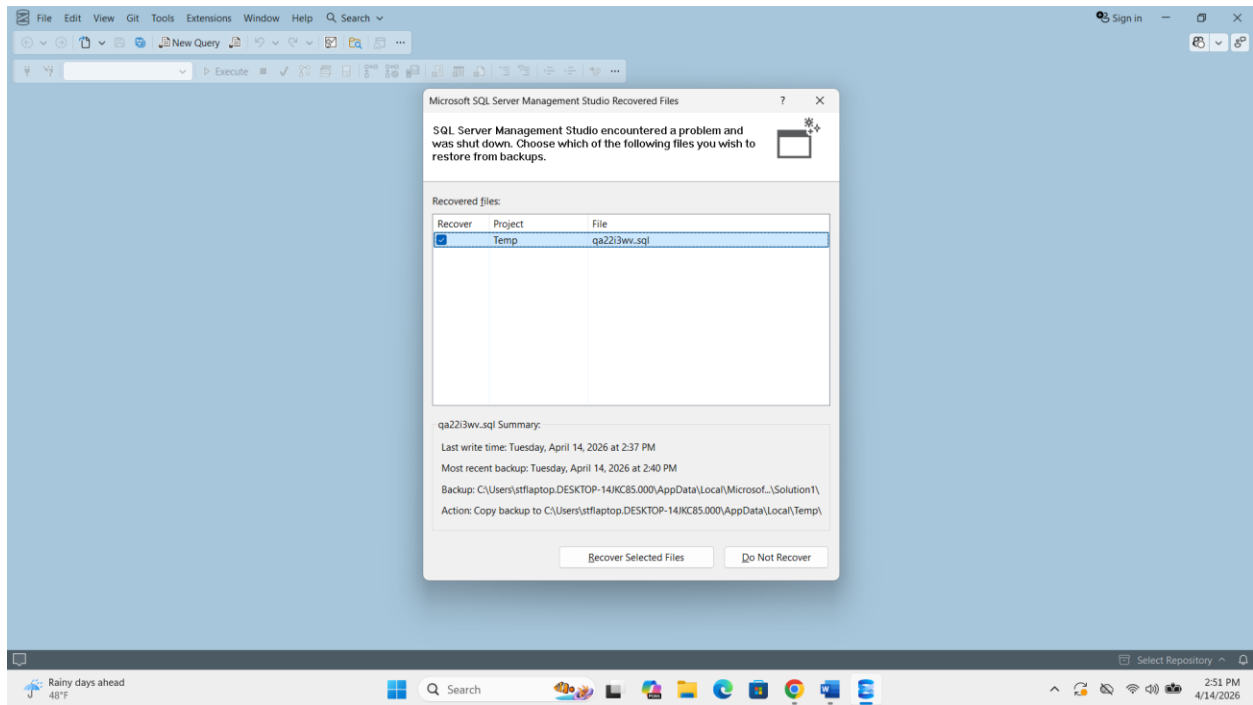
Time when it happens and I screenshot: 13:57:43pm



Time crash when it happens and I screenshot: 14:25:59pm



Time crash when it happens and I screenshot: 14:51:05pm



Time crash when it happens and I screenshot: 14:51:16